Алгоритъм за минимизация

От предходната лекция стана ясно, че задачата за минимизация на даден краен детерминиран автомат $\mathcal{A} = \langle \Sigma, Q, q_{\texttt{start}}, \delta, F \rangle$ без недостижими състояния се свежда до намирането на класовете от еквивалентни състояния, т.е. класовете на еквивалентност на релацията $\equiv_A$.

Това ще направим постъпково като апроксимираме $\equiv_{\mathcal{A}}$. Да напомним, че за две състояния $p, q \in Q$:

$$p \equiv_{\mathcal{A}} q \iff \mathcal{L}_{\mathcal{A}}(p) = \mathcal{L}_{\mathcal{A}}(q) \iff \forall \omega \in \Sigma^\star \Big(\delta^\star(p, \omega) \in F \Leftrightarrow \delta^\star(q, \omega) \in F \Big).$$

Кванторът за общност е върху безкрайното множество $\Sigma^\star$. Ще поискаме да го ограничим до $\Sigma^{\leq n} = \Sigma^0 \cup \Sigma^1 \cup \ldots \cup \Sigma^n$. Така ще дефинираме

$$p \equiv^n_{\mathcal{A}} q \iff \forall \omega \in \Sigma^{\leq n} \Big(\delta^\star(p, \omega) \in F \Leftrightarrow \delta^\star(q, \omega) \in F \Big).$$

Забележете, че при фиксирано n ние можем да проверим за всички думи в $\Sigma^{\leq n}$ дали е в сила съответното условие, т.е. по дадени n, p и q ние имаме алгоритъм, който проверява дали $p \equiv^n_{\mathcal{A}} q$. Имаме $\Sigma^{\leq n} \subset \Sigma^{\leq n+1}$, така че $p \equiv^{n+1}_{\mathcal{A}} q$ влече $p \equiv^n_{\mathcal{A}} q$, т.е. получаваме следната намаляваща редица от релации:

$$\equiv^0_{\mathcal{A}} \supseteq \equiv^1_{\mathcal{A}} \supseteq \ \ldots \ \supseteq \equiv^n_{\mathcal{A}} \supseteq \equiv^{n+1}_{\mathcal{A}} \supseteq \ \ldots$$

Също така е ясно, че $\equiv_{\mathcal{A}}$ съвпада със сечението на всички $\equiv^n_{\mathcal{A}}$, т.е. като множества от наредени двойки

$$\equiv_{\mathcal{A}} \ = \ \bigcap_{n=0}^{\infty} \equiv^n_{\mathcal{A}}$$

или още: $p \equiv_{\mathcal{A}} q$ тогава и само тогава, когато $p \equiv^n_{\mathcal{A}} q$ за всяко n.

Идеята на алгоритъма ще бъде последователно да изчисляваме всяка от релациите $\equiv^n_{\mathcal{A}}$ със следното рекурентно свойство, което показва как $\equiv^{n+1}_{\mathcal{A}}$ се изразява чрез $\equiv^n_{\mathcal{A}}$:

$$p \equiv^{n+1}_{\mathcal{A}} q \quad \iff \quad p \equiv^n_{\mathcal{A}} q \ \ \& \ \ \forall x \in \Sigma \ \delta(p, x) \equiv^n_{\mathcal{A}} \delta(q, x).$$

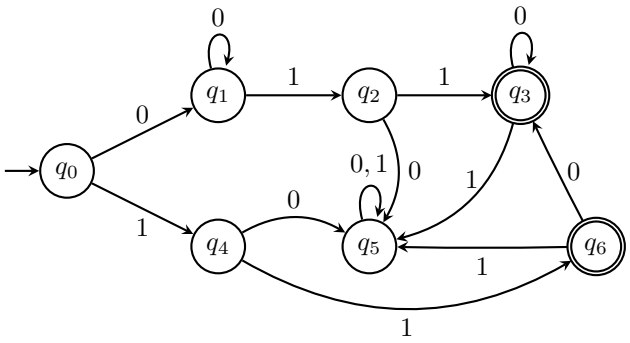
Доказва се лесно с разсъждение, подобно на това в теста на едната буква.

Започваме с $\equiv^0_{\mathcal{A}}$, която има два класа на еквивалентност: F и $Q \setminus F$, тъй като $p \equiv^0_{\mathcal{A}} q$ (при $\omega = \epsilon$) точно когато $p \in F, \ q \in F$ или $p \notin F, \ q \notin F$.

Въпросът е на коя итерация да приключим. Тъй като редицата от релациите на еквивалентност $\equiv^n_{\mathcal{A}}$ е намаляваща относно включването, то индексът на $\equiv^n_{\mathcal{A}}$ (т.е. броят на класовете на еквивалентност) не намалява с нарастването на n. Освен това, индексът на една релация на еквивалентност в Q не може да надвишава броя $|Q|$ на състоянията. Затова ще има момент n, в който $\equiv^n_{\mathcal{A}} = \equiv^{n+1}_{\mathcal{A}}$, т.е. когато индексът ще се запази и няма да нарасне (и този момент е най-много $n = |Q| - 1$, тъй като започваме от

индекс 2). Лесно се съобразява, че редицата по-нататък се стабилизира, т.е. $\equiv_{\mathcal{A}}^{n} = \equiv_{\mathcal{A}}^{n+1} = \equiv_{\mathcal{A}}^{n+2} = \ \ldots$, което разбира се означава, че сечението $\equiv_{\mathcal{A}}$ е точно тази последна релация $\equiv_{\mathcal{A}}^{n}$.

Пример за минимизация на автомат. Нека да минимизираме следния автомат $\mathcal{A}$:



Създаваме таблица за релацията $\equiv_{\mathcal{A}}^{0}$. В нея има само два класа състояния: A_0 от нефиналните и A_1 от финалните състояния. Със символа $\downarrow$ указваме в кой клас е началното състояние q_0. Със символа $\star$ маркираме класовете от финални състояния. За всяко състояние q и буква $x \in \{0, 1\}$ в съответната клетка записваме класа, който съдържа състоянието $\delta(q, x)$ съобразно дадения автомат $\mathcal{A}$.

$\equiv_{\mathcal{A}}^{0}$	$\downarrow A_0$					$\star A_1$	
Q	q_0	q_1	q_2	q_4	q_5	q_3	q_6
0	A_0	A_0	A_0	A_0	A_0	A_1	A_1
1	A_0	A_0	A_1	A_1	A_0	A_0	A_0

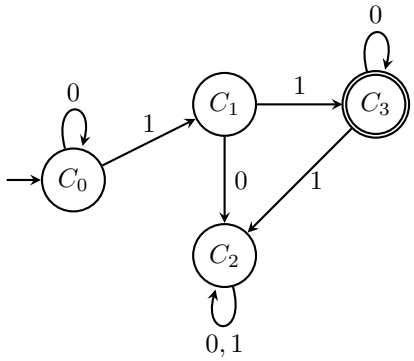
Две състояния са $\equiv_{\mathcal{A}}^{1}$-еквивалентни точно когато са в един и същи клас и имат еднакви стълбове в таблицата за $\equiv_{\mathcal{A}}^{0}$. Така класът A_0 се разпада на двата класа $B_0 = \{q_0, q_1, q_5\}$ и $B_1 = \{q_2, q_4\}$. Класът $A_1 = B_2$ не се разпада. След това продължаваме по същия начин.

$\equiv_{\mathcal{A}}^{1}$	$\downarrow B_0$			B_1		$\star B_2$	
Q	q_0	q_1	q_5	q_2	q_4	q_3	q_6
0	B_0	B_0	B_0	B_0	B_0	B_2	B_2
1	B_1	B_1	B_0	B_2	B_2	B_0	B_0

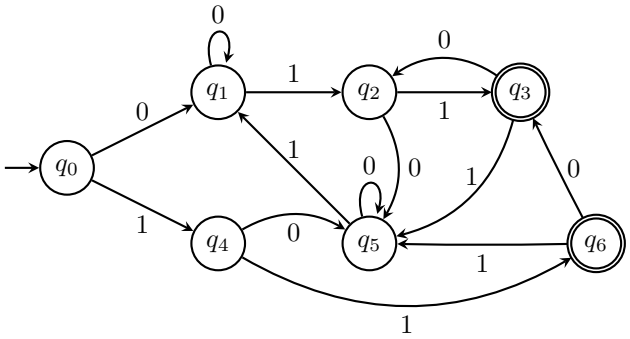
Класът B_0 се разпада на $C_0 = \{q_0, q_1\}$ и $C_2 = \{q_5\}$. Класовете $B_1 = C_1$ и $B_2 = C_3$ се запазват.

$\equiv_{\mathcal{A}}^{2}$	$\downarrow C_0$		C_1		C_2	$\star C_3$	
Q	q_0	q_1	q_2	q_4	q_5	q_3	q_6
0	C_0	C_0	C_2	C_2	C_2	C_3	C_3
1	C_1	C_1	C_3	C_3	C_2	C_2	C_2

На този етап няма повече разпадания, така че алгоритъмът приключва и $\equiv_{\mathcal{A}} = \equiv^2_{\mathcal{A}}$. От последната таблица лесно можем да разчетем преходите в минималния автомат: от класа C_0 с буква 0 се отива в C_0, от класа C_1 с буква 0 се отива в C_2 и т.н. Получаваме като отговор:



За домашно минимизирайте следния автомат $\mathcal{A}$:



Груба оценка на сложността на представения алгоритъм е $O(|Q|^3)$, т.е кубична спрямо броя на състоянията на оригиналния автомат. Броят на итерациите на алгоритъма е най-много $|Q|$ и на всяка итерация попълването на съответната таблица и образуването на новото разбиване изисква време $O(|Q|^2)$. С подходящи структури от данни последното време може да бъде намалено до $O(|Q|)$ и така общото време да стане $O(|Q|^2)$.

Неограничени граматики

Дефиниция 1 Неограничена граматика *е наредена четворка от вида*

$$G = (V, \Sigma, R, S),$$

където:

- V *е крайно множество от* променливи *(нетерминали);*
- Σ *е крайно множество от* букви *(терминали), като* $\Sigma \cap V = \emptyset$*;*
- $R \subseteq (V \cup \Sigma)^+ \times (V \cup \Sigma)^\star$ *е крайно множество от* правила*;*
- $S \in V$ *е началната променлива (аксиома).*

За по-добра яснота, обикновено правилата $(\alpha, \beta) \in R$ ще означаваме като $\alpha \to_G \beta$. Когато е ясно за коя граматика говорим, ще пишем просто $\alpha \to \beta$. Забележете, че лявата част α на едно правило не може да е празната дума. За дясната част β няма такова ограничение.

Идеята на едно правило $\alpha \to \beta$ е, че то позволява заместване на α с β. По-прецизно, дефинираме релацията $\Rightarrow_G$ за *едностъпков извод* по следния начин:

$$[(\alpha, \beta) \in R \ \ \& \ \ \lambda, \rho \in (V \cup \Sigma)^\star] \quad \text{влече} \quad \lambda\alpha\rho \Rightarrow_G \lambda\beta\rho.$$

Така от една дума γ за една стъпка може да се изведе думата γ' с помощта на правилото $\alpha \to_G \beta$ точно когато $\gamma = \lambda\alpha\rho$, т.е. γ съдържа α като поддума и γ' се получава от γ със замяна на тази поддума с β, т.е. $\gamma' = \lambda\beta\rho$.

Свойство 1. Да съобразим, че имаме:

$$\alpha \Rightarrow_G \beta \ \ \& \ \ \lambda, \rho \in (V \cup \Sigma)^\star \quad \text{влече} \quad \lambda\alpha\rho \Rightarrow_G \lambda\beta\rho.$$

Наистина, $\alpha \Rightarrow_G \beta$ означава, че $\alpha = \lambda'\alpha'\rho'$ и $\beta = \lambda'\beta'\rho'$, където $\alpha' \to_G \beta'$ е правило в G. Но тогава $\lambda\alpha\rho = \lambda\lambda'\alpha'\rho'\rho$ и $\lambda\beta\rho = \lambda\lambda'\beta'\rho'\rho$, така че от дефиницията за едностъпков извод имаме $\lambda\alpha\rho \Rightarrow_G \lambda\beta\rho$.

По-нататък да дефинираме *многостъпков извод* на думата β от думата α в граматиката G, което ще означаваме като $\alpha \overset{\ell}{\Rightarrow}_G \beta$ с индукция по броя на стъпките ℓ по следния начин:

- За произволна дума $\alpha \in (V \cup \Sigma)^\star$ е вярно $\alpha \overset{0}{\Rightarrow}_G \alpha$.
- Ако $\alpha \Rightarrow_G \gamma$ и $\gamma \overset{\ell}{\Rightarrow}_G \beta$, то $\alpha \overset{\ell+1}{\Rightarrow}_G \gamma$.

С други думи, при $\ell > 0$, $\alpha \overset{\ell}{\Rightarrow}_G \beta$ точно когато

$$\alpha \Rightarrow_G \gamma_1 \Rightarrow_G \gamma_2 \ldots \Rightarrow_G \gamma_{\ell-1} \Rightarrow_G \beta$$

за някои думи $\gamma_1, \ldots, \gamma_{\ell-1} \in (V \cup \Sigma)^\star$.

Пример 2 *Да разгледаме граматиката G зададена с правилата*

$$S \to_G aSb \mid \varepsilon.$$

Това означава, че $V = \{S\},\ \Sigma = \{a, b\}$ и $R = \{(S, aSb), (S, \varepsilon)\}$.

Обичайно е променливите (нетерминалите) да се означават с главни букви, възможно с индекси, а буквите (терминалите) да се означават с малки букви. Също сме използвали съкращение с $|$, когато задаваме правила с една и съща лява част.

В тази граматика изводите, които започват с аксиомата S имат вида:

$$S \Rightarrow_G aSb \Rightarrow_G aaSbb \Rightarrow_G \ldots \Rightarrow_G a^nSb^n \Rightarrow_G a^nb^n.$$

Някои други примери за изводи в G:

$$aSbSb \Rightarrow_G aaSbbSb,\ \ aSbSb \overset{2}{\Rightarrow}_G aaSbbaSbb,\ \ aSSb \overset{2}{\Rightarrow}_G ab.$$

Също така, $aSSb \not\Rightarrow_G ab$, тъй като без значение кое от двете правила приложим към някой от двата символа S, поне едно S ще се запази.

Дефиниция 3 *Релацията $\overset{\star}{\Rightarrow}_G$ за общ извод дефинираме така:*

$$\alpha \overset{\star}{\Rightarrow}_G \beta \quad \overset{\text{деф}}{\Leftrightarrow} \quad (\exists \ell \in \mathbb{N})[\ \alpha \overset{\ell}{\Rightarrow}_G \beta\ ]$$

Езикът, който се поражда от граматиката G дефинираме с:

$$\mathcal{L}(G) \overset{\text{деф}}{=} \{\omega \in \Sigma^\star \mid S \overset{\star}{\Rightarrow}_G \omega\}.$$

Разбира се, $\overset{\star}{\Rightarrow}_G$ е рефлексивно и транзитивно затваряне на релацията $\Rightarrow_G$. Обръщаме внимание, че в езика $\mathcal{L}(G)$ попадат само думи от $\Sigma^\star$. Променливите от V имат само спомагателна роля при извеждането на думи.

От по-горе е ясно, че езикът на примерната G е $\mathcal{L}(G) = \{a^nb^n \mid n \in \mathbb{N}\}$.

Свойство 2. $\alpha \overset{\ell}{\Rightarrow} \beta$ влече $\lambda\alpha\rho \overset{\ell}{\Rightarrow} \lambda\beta\rho$.

Имаме $\alpha \Rightarrow_G \gamma_1 \Rightarrow_G \gamma_2 \ldots \Rightarrow_G \gamma_{\ell-1} \Rightarrow_G \beta$ и тогава от Свойство 1: $\lambda\alpha\rho \Rightarrow_G \lambda\gamma_1\rho \Rightarrow_G \lambda\gamma_2\rho \ldots \Rightarrow_G \lambda\gamma_{\ell-1}\rho \Rightarrow_G \lambda\beta\rho$. Формалното доказателство може да се проведе с индукция по ℓ.

Свойство 3. $\alpha \overset{\ell_1}{\Rightarrow} \beta$, $\beta \overset{\ell_2}{\Rightarrow} \gamma$ влече $\alpha \overset{\ell_1+\ell_2}{\Rightarrow} \gamma$.

Идеята е просто да съединим двете последователности от едностъпкови изводи. Формалното доказателство е с индукция по ℓ_2.

Свойство 4. $\alpha_1 \overset{\ell_1}{\Rightarrow} \beta_1$, $\alpha_2 \overset{\ell_2}{\Rightarrow} \beta_2$ влече $\alpha_1\alpha_2 \overset{\ell_1+\ell_2}{\Rightarrow} \beta_1\beta_2$.

От Свойство 2: $\alpha_1\alpha_2 \overset{\ell_1}{\Rightarrow} \beta_1\alpha_2$ и също $\beta_1\alpha_2 \overset{\ell_2}{\Rightarrow} \beta_1\beta_2$. Остава да приложим Свойство 3.

В лекцията продължихме с класификация на граматиките по отношение на вида на техните правилата.

Контекстни граматики

Казваме, че $G = (V, \Sigma, R, S)$ е *контекстна граматика*, ако правилата на G са от вида $\lambda A \rho \to \lambda \alpha \rho$, където $A \in V$, $\lambda, \rho \in (V \cup \Sigma)^\star$ и $\alpha \in (V \cup \Sigma)^+$. По този начин, едно правило позволява заместване на A с α, но само ако преди A стои думата λ и след A стои думата ρ. Често думите λ и ρ се разглеждат като *контекст*, който позволява да се осъществи съответната замяна. Затова тези граматики наричаме още *контекстнозависими* (на английски context-sensitive).

Няма да изследваме подробно контекстните граматики. Ще се задоволим само с един пример.

Пример 4 *Езикът $L = \{a^n b^n c^n \mid n > 0\}$ е контекстен, т.е. L се поражда от контекстна граматика.*

Разглеждаме контекстната граматика G зададена със следните правила:

$$
\begin{aligned}
&S \to aSBC \mid aBC\\
&CB \to CZ\\
&CZ \to WZ\\
&WZ \to WC\\
&WC \to BC\\
&aB \to ab\\
&bB \to bb\\
&bC \to bc\\
&cC \to cc.
\end{aligned}
$$

С помощта на първото правило можем да генерираме думата $a^n(BC)^n$. След това сортираме B-тата и C-тата, като използваме $CB \overset{4}{\Rightarrow}_G BC$. Правилото $CB \to BC$ само по себе си не е контекстно, т.е. няма вида, който се изисква в контекстните граматики. Затова се налага да го симулираме, като използваме двете спомагателни променливи Z и W. Достигнали сме до думата $a^n B^n C^n$ и точно в този момент трябва да започне заместването, първо на B с b и след това на C с c. Съобразете, че за тази цел не можем да използваме правила от вида $B \to_G b$ или $C \to_G c$.

Следният извод показва, че $L \subseteq \mathcal{L}(G)$:

$$S \overset{n}{\Rightarrow}_G a^n(BC)^n \overset{\star}{\Rightarrow}_G a^n B^n C^n \overset{n}{\Rightarrow}_G a^n b^n C^n \overset{n}{\Rightarrow}_G a^n b^n c^n.$$

Другата посока $\mathcal{L}(G) \subseteq L$, която показва че G не извежда некоректни думи е доста по-сложна и затова я пропускаме. Накратко, трябва да се покаже, че при извеждане на дума от $\Sigma^\star$ в граматиката G, второто, третото, четвъртото и петото правило не могат да се използват по друг начин, освен да симулират заместване на CB с BC.

С подобна идея може да се покаже, че всяка граматика, в която правилата са несъкращаващи, т.е. имат вида $\alpha \to \beta$, където $|\alpha| \leq |\beta|$, е еквивалентна на контекстна граматика.

Регулярни граматики

Една граматика $G = (V, \Sigma, R, S)$ е *регулярна*, ако правилата в нея имат вида $A \to aB$ и $A \to \epsilon$, където $A, B \in V$ и $a \in \Sigma$.

Забележете, че изводите в такава граматика, започващи с променлива и завършващи с дума от $\Sigma^\star$ задължително имат следния вид:

$$A \;\Rightarrow\; aA_1 \;\Rightarrow\; abA_2 \;\Rightarrow\; \ldots \;\Rightarrow\; ab\ldots cA_n \;\Rightarrow\; ab\ldots c.$$

Във всяка от изведените думи участва точно една променлива в края на думата. Последното приложено правило трябва да има дясна част ε.

Изобразеният извод наподобява изпълнение на краен автомат, ако си мислим, че променливите отговарят на състояния и всяко приложение на правило в извода (без последното) отговаря на прочитане на буква и преминаване в следващо състояние. Това е идеята в следващите два резултата.

Твърдение 5 *За всяка регулярна граматика G съществува НКА $\mathcal{N}$, такъв че $L(\mathcal{N}) = \mathcal{L}(G)$.*

Доказателство. Дадена е регулярна граматика $G = (V, \Sigma, R, S)$. Да номерираме множеството на променливите $V = \{A_0, A_1, \ldots, A_m\}$, така че $S = A_0$. Построяваме НКА $\mathcal{N} = \langle \Sigma, Q, Q_{\texttt{start}}, \Delta, F \rangle$ по следния начин:

- $Q = \{q_0, q_1, \ldots, q_m\}$;
- $Q_{start} = \{q_0\}$;
- $\Delta(q_i, x) = \{q_j \mid A_i \to_G xA_j\}$;
- $F = \{q_i \mid A_i \to_G \epsilon\}$.

Разбира се, автоматът е недетерминиран, тъй като няма ограничения за правилата с една и съща променлива в лявата част. За да проверим коректността на конструкцията, първо ще докажем

$$q_j \in \Delta^\star(\{q_i\}, \alpha) \;\leftrightarrow\; A_i \stackrel{\star}{\Rightarrow}_G \alpha A_j$$

с индукция по думата α.

- Нека $\alpha = \epsilon$. Имаме $q_j \in \Delta^\star(\{q_i\}, \epsilon)$ точно когато $q_i = q_j$, т.е. $i = j$. Изводът $A_i \stackrel{\star}{\Rightarrow}_G A_j$ е възможен само при $i = j$ и дължина 0.
- Нека $\alpha = \beta x$, където $x \in \Sigma$. Тогава:

$$\begin{aligned} q_j \in \Delta^\star(\{q_i\}, \beta x) \;&\leftrightarrow\; \exists k \leq m\; q_k \in \Delta^\star(\{q_i\}, \beta) \;\;\&\;\; q_j \in \Delta(q_k, x) \\ &\leftrightarrow\; \exists k \leq m\; A_i \stackrel{\star}{\Rightarrow}_G \beta A_k \;\;\&\;\; q_j \in \Delta(q_k, x) \\ &\leftrightarrow\; \exists k \leq m\; A_i \stackrel{\star}{\Rightarrow}_G \beta A_k \;\;\&\;\; A_k \to_G xA_j \\ &\leftrightarrow\; A_i \stackrel{\star}{\Rightarrow}_G \beta x A_j. \end{aligned}$$

Използвахме последователно: дефиницията на $\Delta^\star$, индукционното предположение, дефиницията на Δ и накрая горните съображения за вида на изводите в G. Накрая:

$$\begin{aligned}
\alpha \in L(\mathcal{N}) &\leftrightarrow \Delta^\star(\{q_0\}, \alpha) \cap F \neq \emptyset \\
&\leftrightarrow \exists q_j \in F,\ \ q_j \in \Delta^\star(\{q_0\}, \alpha) \\
&\leftrightarrow \exists q_j \in F,\ \ A_0 \stackrel{\star}{\Rightarrow}_G \alpha A_j \\
&\leftrightarrow \exists A_j \in V,\ \ A_j \to_G \epsilon\ \ \&\ \ A_0 \stackrel{\star}{\Rightarrow}_G \alpha A_j \\
&\leftrightarrow S \stackrel{\star}{\Rightarrow}_G \alpha\ \leftrightarrow\ \alpha \in \mathcal{L}(G).
\end{aligned}$$

Твърдение 6 *За всеки ДКА $\mathcal{A}$ съществува регулярна граматика G, такава че $\mathcal{L}(G) = L(\mathcal{A})$.*

Доказателство. При даден ДКА $\mathcal{A} = (\Sigma, Q, q_{start}, \delta, F)$ номерираме състоянията $Q = \{q_0, q_1, \ldots, q_m\},\ q_0 = q_{start}$ и построяваме регулярна граматика $G = (V, \Sigma, R, S)$, където $V = \{A_0, A_1, \ldots, A_m\}$, $S = A_0$ и

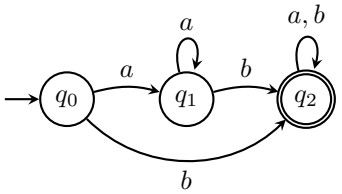
$$R\ =\ \{A_i \to_G xA_j \mid \delta(q_i, x) = q_j\}\ \cup\ \{A_i \to_G \epsilon \mid q_i \in F\}.$$

Както в предходното твърдение се показва, че $\delta^\star(q_i, \alpha) = q_j\ \leftrightarrow\ A_i \stackrel{\star}{\Rightarrow}_G \alpha A_j$ и като следствие се извежда, че $S \stackrel{\star}{\Rightarrow}_G \alpha\ \leftrightarrow\ \delta^\star(q_0, \alpha) \in F$, т.е. $\mathcal{L}(G) = L(\mathcal{A})$.

Забележете, че конструкцията в последното твърдение може да се приложи и за недетерминиран автомат. Единственият проблем е, че по дефиниция НКА може да има повече от едно начално състояние. Затова се добавя нова аксиома S и по едно правило $S \to_G \alpha$ за всяко правило $A_j \to_G \alpha$ с лява част, съответна на начално състояние q_j. По този начин се симулират правила от вида $S \to_G A_j$ (наричат се преименуващи).

Комбинацията от двете твърдения показва, че един език L е регулярен тогава и само тогава, когато L се поражда от регулярна граматика.

Пример. Разглеждаме следния автомат:



Регулярна граматика G за езика $\mathcal{L}(\mathcal{A})$ можем да дефинираме така:

- $\Sigma = \{a, b\}$;
- На всяко състояние q_i на автомата ще съответства променливата A_i, т.е. $V = \{A_0, A_1, A_2\}$;

- Началната променлива е A_0, защото q_0 е началното състояние на автомата;
- Правилата следват дефиницията на δ функцията:

$$A_0 \to aA_1 \mid bA_2$$
$$A_1 \to aA_1 \mid bA_2$$
$$A_2 \to aA_2 \mid bA_2 \mid \varepsilon.$$

Материалът за седмата лекция от записките на Вътев:
глава 2.10 (**Кубичен алгоритъм за минимизация**), страници 82-85,
глава 3.1 (**Неограничени граматики**), страници 98-101,
глава 3.2 (**Контекстни граматики**), страница 102,
глава 3.3 (**Регулярни граматики**), страници 103-104.

За допълнително четене от учебника на Hopcroft, Motwani, Ullman:
глава 4.4 (Equivalence and Minimization of Automata).